



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

FASTIFY PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on performance, plugin system, schema validation, and hooks

TOTAL: 10 QUESTIONS

Q1 What makes Fastify faster than Express.js?

Threat Model

Q6 How does Fastify's response serialization work?

Observability

Q2 How does Fastify's plugin system and encapsulation work?

Architecture

Q7 How do you add security headers and rate limiting in Fastify?

Lifecycle

Q3 How does Fastify's JSON Schema validation protect routes?

Configuration

Q8 What are Fastify decorators and how do you use them?

Compliance

Q4 What are Fastify lifecycle hooks and when do they run?

Hardening

Q9 How do you integrate PostgreSQL in a Fastify application?

Testing

Q5 How do you implement JWT authentication in Fastify?

SSO / IdP

Q10 How do you write tests for Fastify routes?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Fastify uses a radix-tree (find-my-way) router

Mitigation

Fastify uses a radix-tree (find-my-way) router that is O(1) per segment vs Express's regex iteration. JSON serialization uses fast-json-stringify which generates compiled serializers from JSON Schema, skipping generic JSON.stringify entirely.

A6 Define a response JSON Schema in

Observability

Define a response JSON Schema in the route schema ({schema:{response:{200:{...}}}}). Fastify generates a compiled serializer with fast-json-stringify at startup. This is ~3x faster than JSON.stringify and strips fields not in the schema, preventing data leakage.

A2 fastify.register(plugin, options) adds a plugin in

Primitives

fastify.register(plugin, options) adds a plugin in an encapsulated scope. Decorators and hooks added inside a plugin are scoped to it and its children. fastify-plugin() opts out of encapsulation, making decorators visible to the parent scope.

A7 @fastify/helmet sets security headers (CSP,

Automation

@fastify/helmet sets security headers (CSP, X-Frame-Options). @fastify/rate-limit tracks requests per IP using in-memory or Redis counters and returns 429 when limits are exceeded. Both are registered as plugins with fastify.register().

A3 Attach a schema object to route

Hardening

Attach a schema object to route options: {schema:{body:{type:'object',required:['name'],properties:{name:{type:'string'}}}}}. Fastify uses ajv to validate incoming requests before the handler runs and returns 400 with a structured error if validation fails.

A8 fastify.decorate('redis', redisClient) adds a

Governance

fastify.decorate('redis', redisClient) adds a property to the Fastify instance. fastify.decorateRequest('user', null) adds a property to every Request object. This allows plugins to attach shared state (DB connections, auth context) accessible in handlers.

A4 Hooks run at named stages: onRequest

Gotchas

Hooks run at named stages: onRequest (first, before parsing), preParsing, preValidation, preHandler (after validation), preSerialization, onSend (before response), onResponse (after). Use fastify.addHook('preHandler', fn) or route-level hooks.

A9 @fastify/postgres wraps pg pool and adds

Assessment

@fastify/postgres wraps pg pool and adds fastify.pg to the instance. For an ORM, use Prisma (prismaClient as a decorator) or Drizzle. Always close connections on shutdown using fastify.addHook('onClose', () => pool.end()).

A5 @fastify/jwt decorates the instance with

Federation

@fastify/jwt decorates the instance with fastify.jwt.sign(payload) and fastify.jwt.verify(token). Register fastify.authenticate as a preHandler hook on protected routes. The plugin reads the Bearer token from the Authorization header automatically.

A10 fastify.inject({method:'POST', url:'/users',

Optimization

fastify.inject({method:'POST', url:'/users', payload:{name:'Alice'}}) sends a request in-process without a TCP connection. It resolves with a LightMyRequest response. Use Jest or tap assertions on response.statusCode and JSON.parse(response.body).